# Introduction to mathematical optimization

INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

datacamp

# Coming up...

## You'll learn about...

- Solving real-world optimization problems
- Build a toolkit to tackle different problems

## What you need to know beforehand...

- NumPy for numerical computation
  - Introduction to NumPy

## What you don't need to know...

- Calculus
- Algorithms

# What is mathematical optimization?

- Finding the *ideal* inputs for a specific problem

- **Examples:**
  - Maximizing crop yields



- Soil conditions, weather
- Best crop quantity and quality

# What is mathematical optimization?



- Optimal distribution routes
- Distance and traffic
- Improve delivery times and reduce costs

# The objective function

- Describes the relationship between input variables and the result

**Furniture manufacture:**

- Maximize profit, P
- Quantity, q

$$P = 40q - 0.5q^2$$

- Which value of q maximizes P?

# Optimize with Python

```
import numpy as np
import matplotlib.pyplot as plt

qs = np.arange(80)

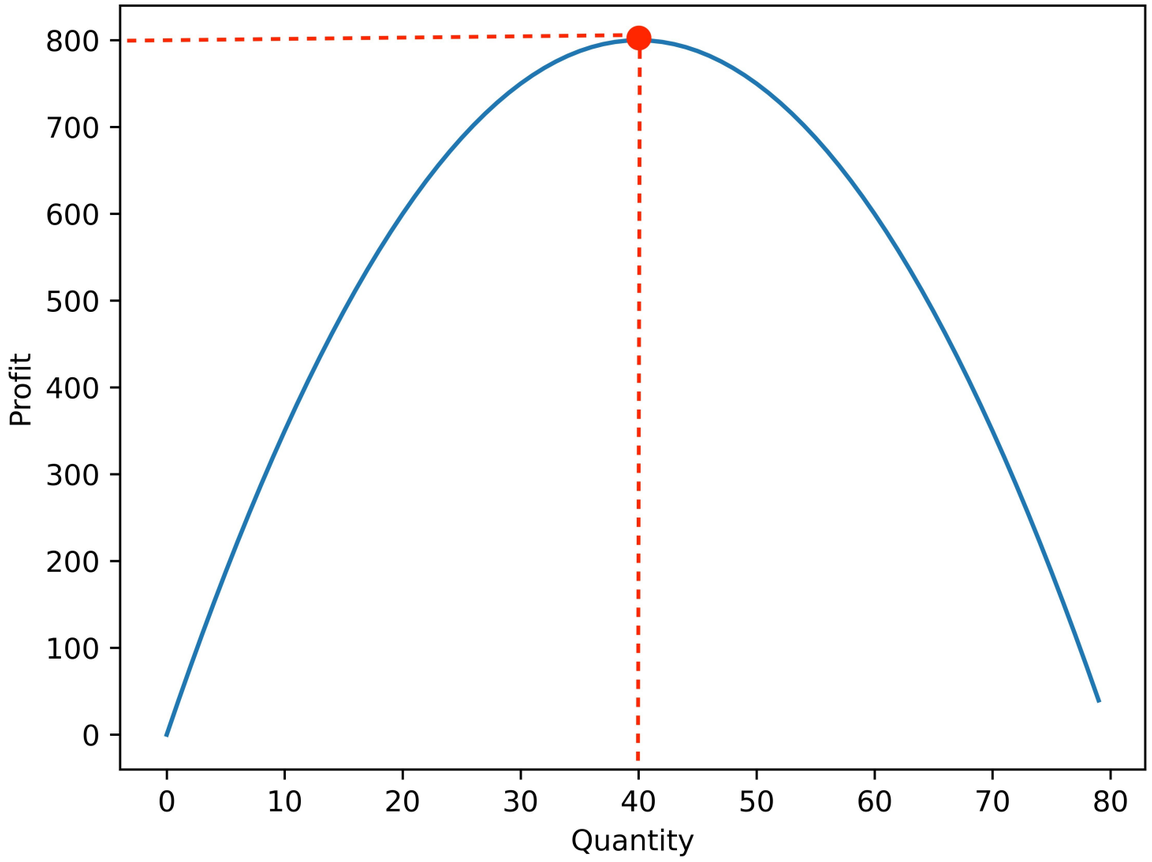
def profit(q):
  return 40 * q - 0.5 * q**2

plt.plot(qs, profit(qs))
plt.xlabel('Quantity')
plt.ylabel('Profit')
plt.show()
```

# Optimization in manufacturing

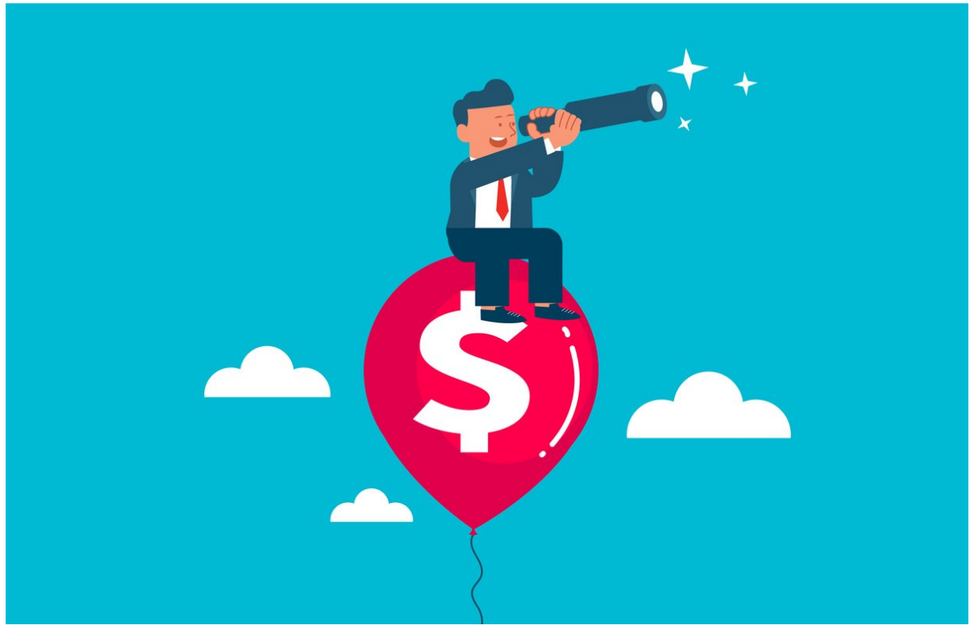
- Maximum value = optimum value

- Possible reasons for the shape:
  - Not enough staff or equipment



# Exhaustive search

- "Brute force" method
- Calculating the profit for a range of quantities
- Select the one that produces the largest profit



# Exhaustive search in Python

```
import numpy as np

qs = np.arange(80)

def profit(q):
  return 40 * q - 0.5 * q**2

profits = profit(qs)
max_profit = profits.max()
max_ind = np.argmax(profits)
q_opt = qs[max_ind]

print(f"The optimum is {q_opt} pieces of furniture, which makes ${max_profit} profit.")
```

```
The optimum is 40 pieces of furniture, which makes $800 profit.
```

# Exhaustive search advantages and disadvantages

- Advantages
  - (+) Simple implementation
  - (+) No need for expensive software
  - (+) Minimal assumptions
- Disadvantages
  - (-) Will not scale for complex cases

# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

# Univariate optimization

INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

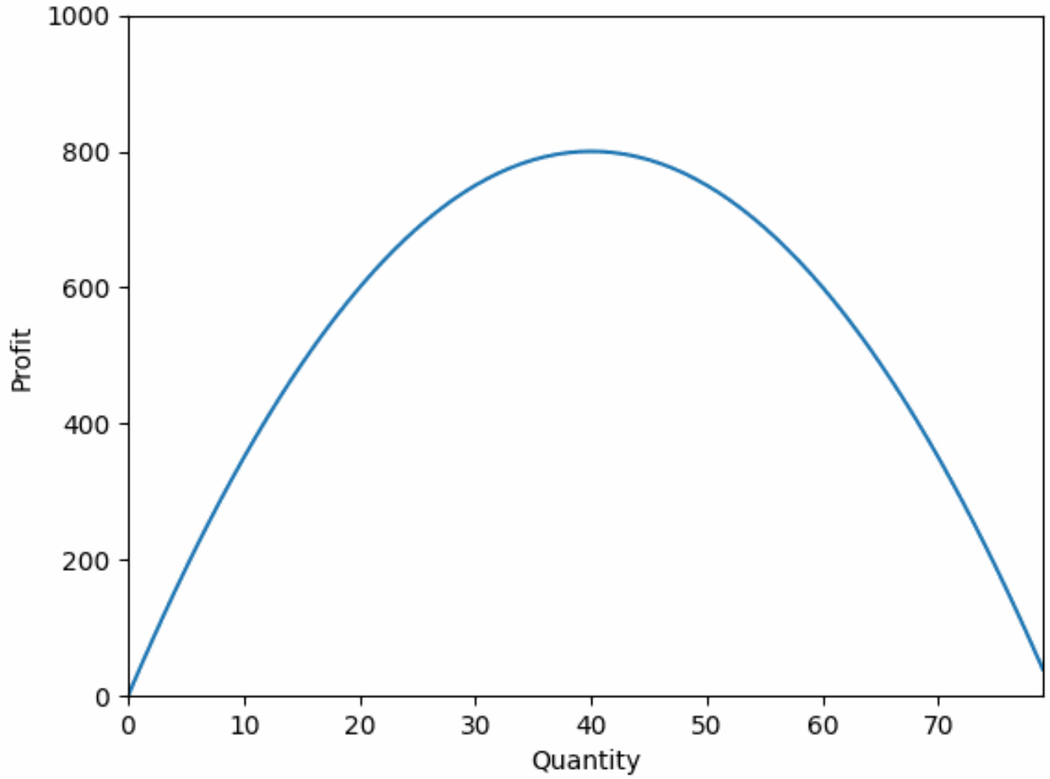
datacamp

# Example: Optimization in manufacturing

**Objective function:**

$$P = 40q - 0.5q^2$$

- **Univariate:** has one variable

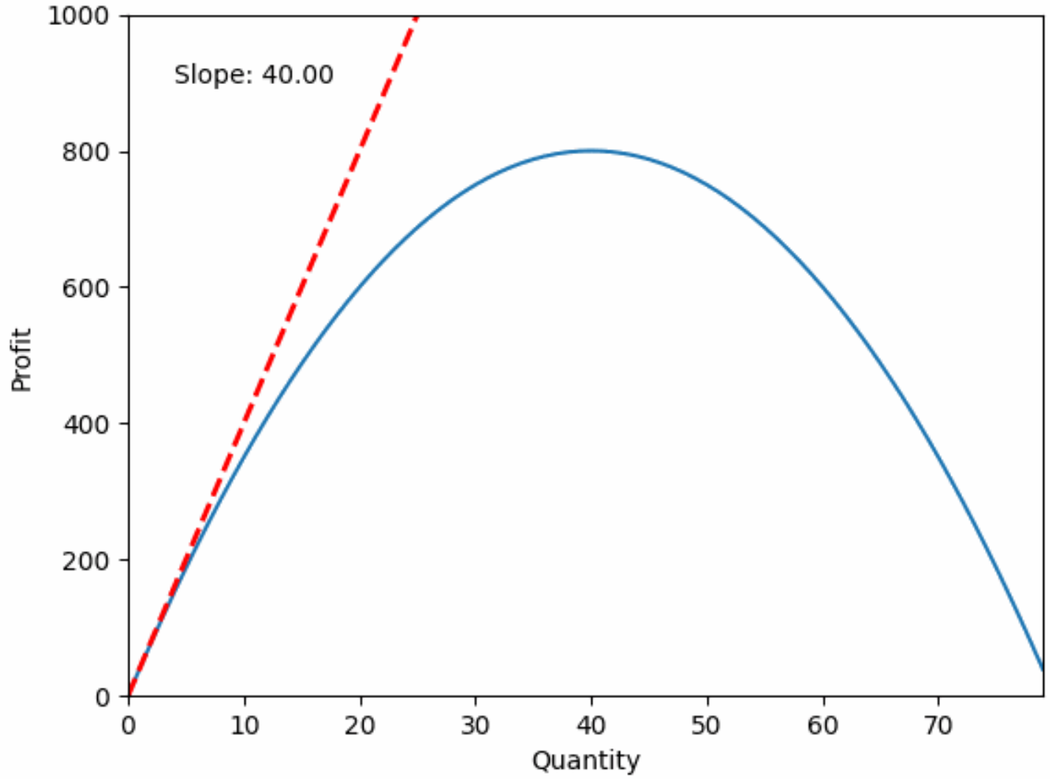


# Example: Optimization in manufacturing

**Objective function:**

$$P = 40q - 0.5q^2$$

- **Univariate:** has a single variable



# Calculating derivatives

**Objective function:**

$$P = 40q - 0.5q^2$$

**Derivative:** describes how the slope behaves

$$\frac{dP}{dq} = 40 - q$$

```
from sympy import symbols, diff, solve

q = symbols('q')
P = 40 * q - 0.5 * q**2
dp_dq = diff(P)
print(f"The derivative is: {dp_dq}")
```

```
The derivative is: 40 - 1.0*q
```

[1] https://docs.sympy.org/latest/index.html

# The critical point

**Optimum is found where the derivative function equals zero**

- **Critical points:** points where the derivative function is zero

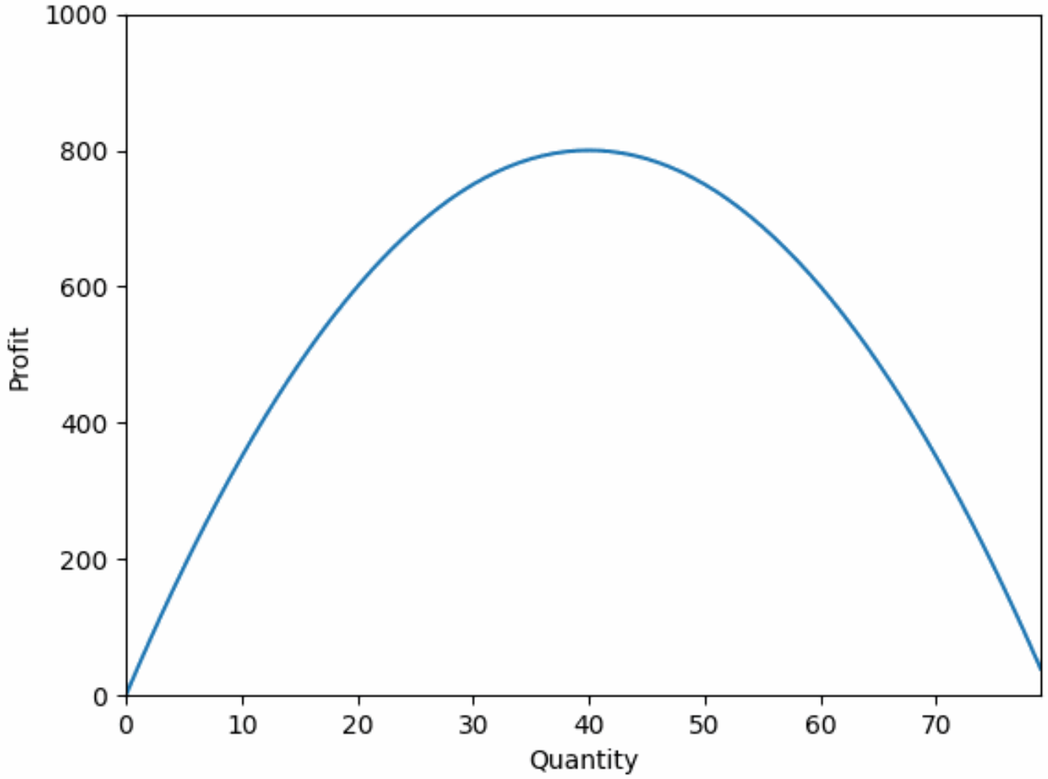
Optimum $q$ satisfies:

$$\frac{dP}{dq} = 40 - q = 0$$

```
q_opt = solve(dp_dq)
print(f"Optimum quantity: {q_opt}")
```

```
Optimum quantity: [40.0000000000000]
```

# Maxima, minima, or neither?



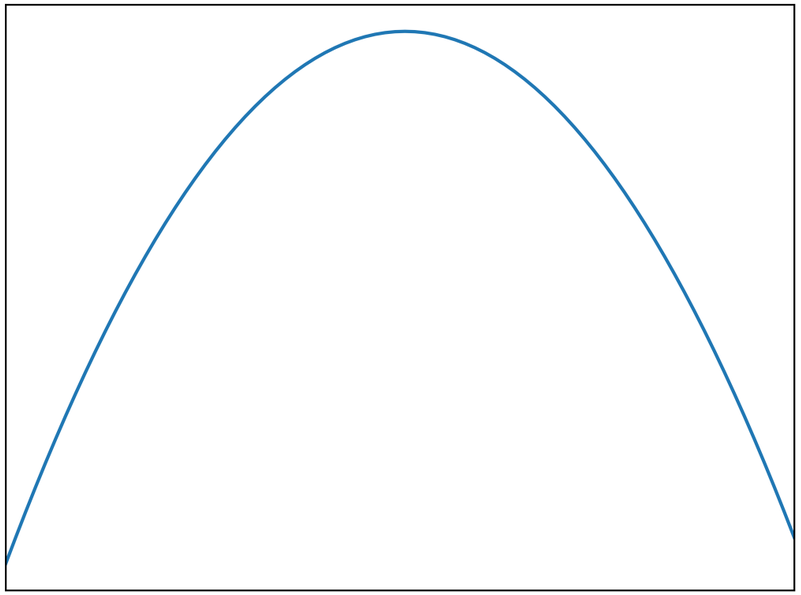

**Objective function:**

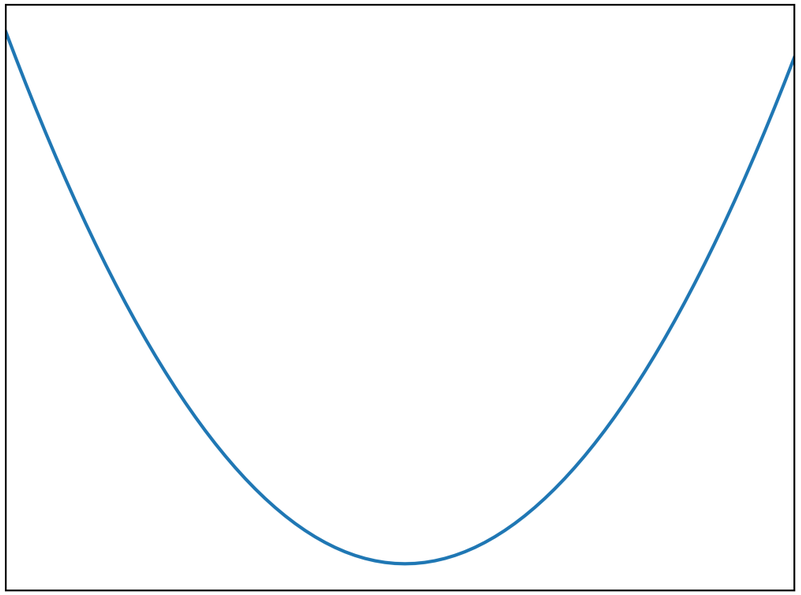
$$p = 40q - 0.5q^2$$

```
q_opt = solve(p_prime)
print(f"Optimum quantity: {q_opt}")
```

```
Optimum quantity: [40.0000000000000]
```

# Convexity and concavity

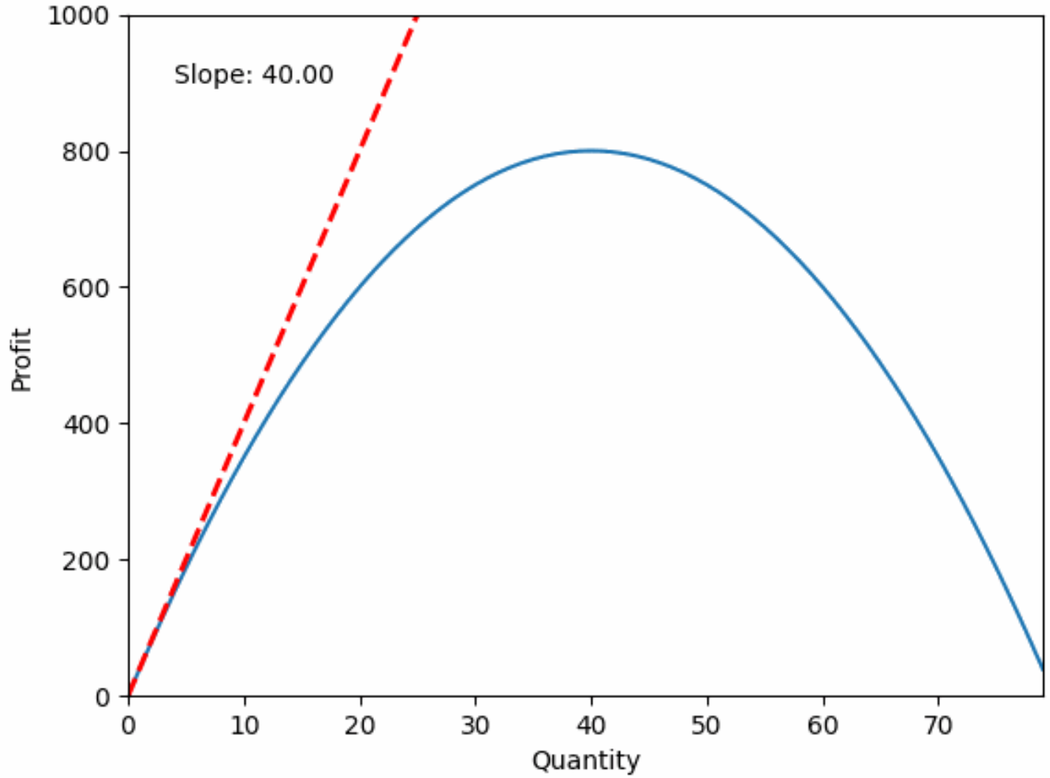



- Maxima
- Minima

# Second derivative

- The derivative of the derivative
- Rate the slope changes as the variable changes




- If 2nd derivative at the point $< 0$: **Maxima**
- If 2nd derivative at the point $> 0$: **Minima**
- If 2nd derivative at point $= 0$: neither

# Example: Maximum, minimum, or neither?

**Derivative:**

$$\frac{dp}{dq} = 40 - q$$

**Second derivative:**

$$\frac{d^2p}{dq^2} = -1 < 0$$

- Maxima!

```
d2p_dq2 = diff(dp_dq)
sol = d2p_dq2.subs('q', q_opt)

print(f"The 2nd derivative is: {sol}")
```

```
The 2nd derivative is: -1.000000000000
```

# Derivatives in optimization

- First derivatives
  - Find **critical points** in optimization problems
- Second derivatives
  - Identify as **minima** or **maxima**

# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

# Multivariate optimization

INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

datacamp

# The biscuit factory

Objective function:

- $F = K^{0.34} \times L^{0.66}$
  - $F$: production function
  - Labor ($L$): hours put in by workers
  - Capital ($K$): hours machines operated
  - **Multivariate** problem



# Partial derivatives

## Univariate

**Objective function:**

$$p = 40q - 0.5q^2$$

**Derivative:** slope of the objective function changes with changes to single variable

$$\frac{dp}{dq} = 40 - q$$

## Multivariate

**Objective function:**

$$F = K^{0.34} \times L^{0.66}$$

**Partial derivatives:** how the slope changes with respect to each variable

$\frac{\partial F}{\partial K}$ and $\frac{\partial F}{\partial L}$

# Solving multivariate problems

Objective function:

- $F = K^{0.34} \times L^{0.66}$
  - $F$: production function
  - Labor ($L$): hours put in by workers
  - Capital ($K$): hours machines operated

```
from sympy import symbols, diff, solve

K, L = symbols('K L')
F = K**.34 * L**.66

dF_dK = diff(F, K)
dF_dL = diff(F, L)
crit_points = solve([dF_dK, dF_dL], (K, L))

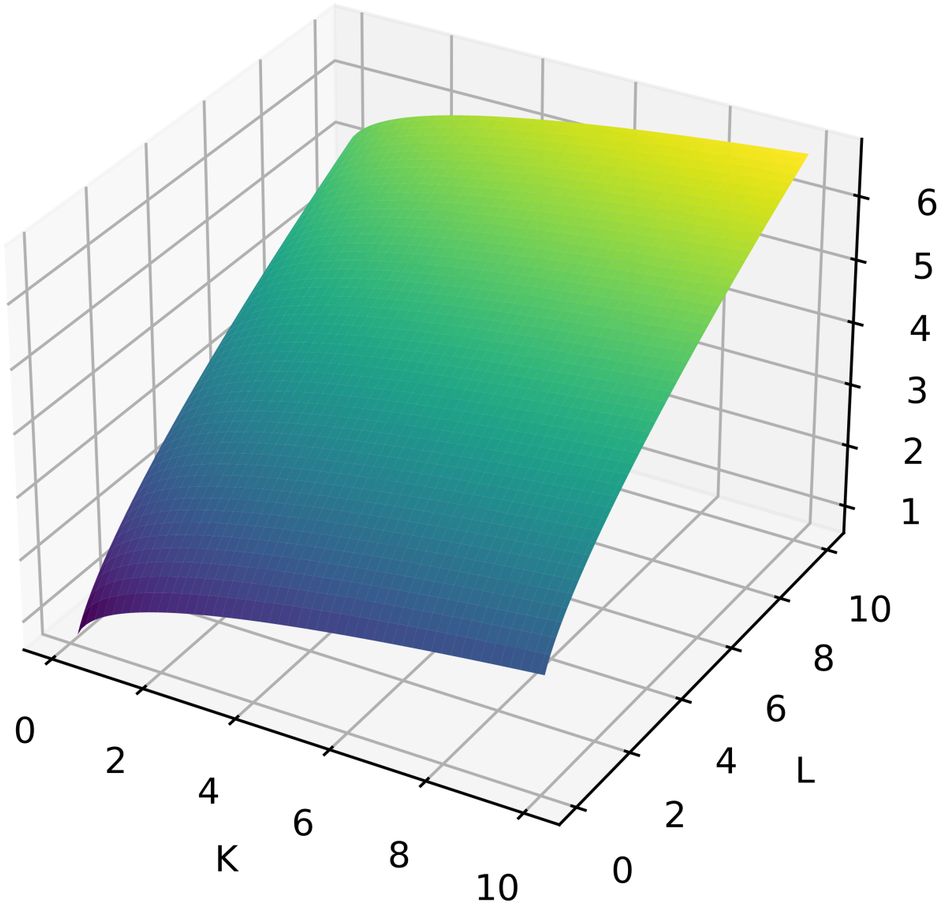
print(crit_points)
```

```
[]
```

# Our production function

Objective function:

- $F = K^{0.34} \times L^{0.66}$
- As $K$ and $L$ increase, $F$ increases
- No **maxima** or **minima**!

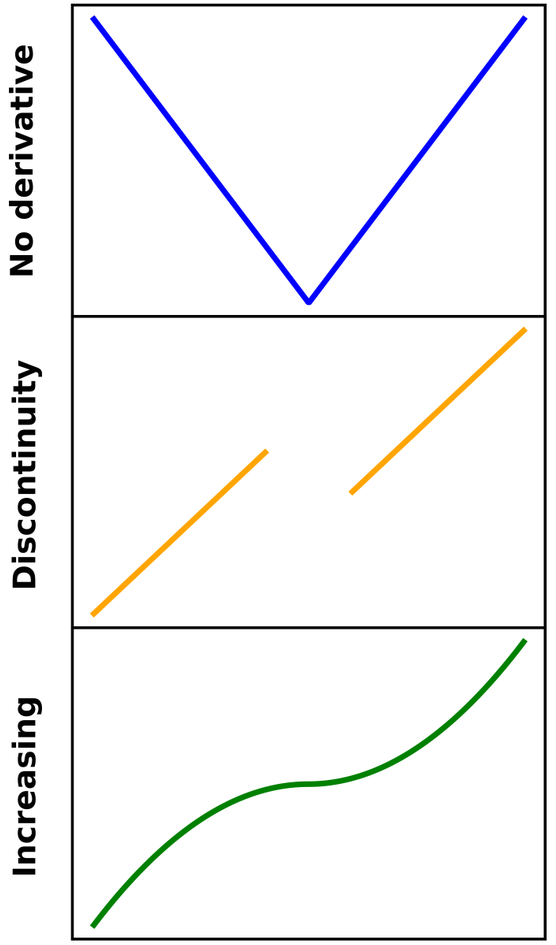


# The limits of differentiation

Watch out for:

- No derivative
- Discontinuities
- Increasing or decreasing functions

- **Non-differentiable**: cannot be optimized using differentiation



# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON